

1. Dependencies VS Dev Dependencies

- **Dependencies:** Packages required when the application runs in production. Examples: Express, React, React Router, etc. Needed to run the application.
- **Dev Dependencies:** Packages needed only during development, testing, linting, or building the project. Examples: ESLint, Vite, testing tools, etc. Needed to develop/build/test the application.

2. Indexing in Db

- Database index used to help the database engine quickly locate and retrieve data without scanning every row in a table.
- **Working:**
 - i. When we create an index on a column, the database creates a separate data structure that stores the indexed values and references to the related rows.
 - ii. When we run a query, the database checks whether an available index can help.
 - iii. If the index is useful, the database uses it to find the required rows faster instead of checking the entire table.
 - iv. Indexes are especially useful for large tables with frequently searched or filtered data.
- **Types**
 - i. **Single column Index:** Indexes a single column. When queries frequently filter or sort by one column.
 - ii. **Composite Index:** An index that includes two or more columns. When queries frequently filter or sort by multiple columns, especially in the same order.
 - iii. **Unique Index:** Ensures that all values in the indexed column(s) are unique. To force data integrity, such as no two users, have the same email.
 - iv. **Full-text Index:** Optimized for searching large text fields. When need to search within text field in a column for word or phrase (e.g Blog post)
 - v. **Spatial Index:** Used for geographic data types. When working with location-based queries.
- **Pros**
 - i. **Faster Queries:** Especially useful for SELECT queries and conditions involving WHERE, JOIN, and sometimes GROUP BY and ORDER BY.
 - ii. **Efficient Data Access:** Helps the database find required records without scanning the entire table.
 - iii. **Improved Sorting:** Appropriate indexes can make some sorting operations more efficient.
 - iv. **Better Performance on Large Tables:** Indexes become more useful as the amount of data increases.
 - v. **Data Integrity:** Unique indexes can prevent duplicate values, such as duplicate email addresses.

- **Cons**

- i. **Increased Storage Space:** Indexes require additional disk space.
- ii. **Slower INSERT, UPDATE, and DELETE:** When data changes, the related indexes also need to be updated.
- iii. **Too Many Indexes Can Reduce Performance:** Creating unnecessary indexes can increase storage and can make write operations slower.
- iv. **Complexity in Choosing the Right Index:** The wrong index may provide little or no benefit, so indexes should be created based on actual queries and database usage.

3. In Which field it will use:

- **Messaging Modal**

- i. Because the Messages table can contain many sender and receiver IDs for different chats. Indexing these fields helps MySQL find messages between specific users faster. Indexing is helpful for large databases in this case.
- ii. **Flow:** Sequelize code -> Message.findAll() -> generate SQL Query -> MySQL receives query -> check index -> find match senderId & receiver Id -> return message.

4. Method to implement Indexing:

- **In modal:**

```
indexes: [  
  {  
    name: "sender_receiver_id",  
    fields: ["senderId", "receiverId"],  
  },  
],
```

Sometimes using: BTREE, BHASH

- **In migration**

- i. Using the queryInterface (Tool or software mechanism that lets a user or program ask a db for specific info: the target table name, specific column to index, an optional configuration block).
- ii. Method=> queryInterface.addIndex(), queryInterface.removeIndex()
- iii. "Up" and "down" in Sequelize migrations represent the forward changes, and rollback changes applied to your database schema.

Query: `EXPLAIN SELECT * FROM message WHERE (senderId=1 AND receiverId=2) OR (senderId=2 AND receiverId=1) ORDER BY createdAt ASC;`

Inserting Data into table:

```
mysql> SELECT * from user;
```

id	email	password	createdAt	updatedAt
1	hadia@gmail.com	123	2026-09-01 13:32:31	2026-09-01 13:32:31
2	hadi@gmail.com	123	2026-09-01 13:32:31	2026-09-01 13:32:31
3	hadiashahid@gmail.com	123	2026-09-01 14:09:51	2026-09-01 14:09:51
4	hadishahid@gmail.com	123	2026-09-01 14:09:51	2026-09-01 14:09:51

4 rows in set (0.00 sec)

```
mysql> SELECT * from message;
```

id	senderId	receiverId	text	createdAt	updatedAt
1	1	2	Hi Hadi, how are you?	2026-09-01 13:34:27	2026-09-01 13:34:27
2	2	1	Hello Hadia! I am doing great, thank you.	2026-09-01 13:34:27	2026-09-01 13:34:27
3	1	2	Are you working on the Node.js project today?	2026-09-01 13:34:27	2026-09-01 13:34:27
4	2	1	Yes, I am setting up the MySQL database models right now.	2026-09-01 13:34:27	2026-09-01 13:34:27
5	1	2	Awesome! Did you fix that timestamps error?	2026-09-01 13:34:27	2026-09-01 13:34:27
6	2	1	Yes, I just added NOW() to the manual insert query and it worked perfectly!	2026-09-01 13:34:27	2026-09-01 13:34:27
7	3	4	Hi Hadi, how are you?	2026-09-01 14:11:52	2026-09-01 14:11:52
8	2	3	Hello Hadia! I am doing great, thank you.	2026-09-01 14:11:52	2026-09-01 14:11:52
9	3	4	Are you working on the Node.js project today?	2026-09-01 14:11:52	2026-09-01 14:11:52
10	4	3	Yes, I am setting up the MySQL database models right now.	2026-09-01 14:11:52	2026-09-01 14:11:52
11	3	2	Awesome! Did you fix that timestamps error?	2026-09-01 14:11:52	2026-09-01 14:11:52
12	4	3	Yes, I just added NOW() to the manual insert query and it worked perfectly!	2026-09-01 14:11:52	2026-09-01 14:11:52

12 rows in set (0.00 sec)

Without Index EXPLAIN:

```
mysql> EXPLAIN SELECT * FROM message WHERE (senderId = 1 AND receiverId = 2) OR (senderId = 2 AND receiverId = 1) ORDER BY createdAt ASC;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	message	NULL	ALL	NULL	NULL	NULL	NULL	12	8.33	Using where; Using filesort

1 row in set, 1 warning (0.01 sec)

```
mysql> EXPLAIN ANALYZE SELECT * FROM message WHERE (senderId = 1 AND receiverId = 2) OR (senderId = 2 AND receiverId = 1) ORDER BY createdAt ASC;
```

```
EXPLAIN
```

```

+-----+
| message.createdAt (cost=1.45 rows=12) (actual time=0.129..0.132 rows=6 loops=1)
| -> Sort: message.createdAt (cost=1.45 rows=12) (actual time=0.129..0.132 rows=6 loops=1)
| -> Filter: ((message.receiverId = 2) and (message.senderId = 1)) or ((message.receiverId = 1) and (message.senderId = 2))) (cost=1.45 rows=12) (actual time=0.0621..0.0963 rows=6 loops=1)
| -> Table scan on message (cost=1.45 rows=12) (actual time=0.0571..0.0847 rows=12 loops=1)
+-----+
1 row in set (0.00 sec)

```

```
mysql> SHOW CREATE TABLE ON message;
```

```
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'ON message' at line 1
```

```
mysql> SHOW CREATE TABLE message;
```

Table	Create Table
message	CREATE TABLE `message` (`id` int NOT NULL AUTO INCREMENT, `senderId` int NOT NULL, `receiverId` int NOT NULL, `text` varchar(255) DEFAULT NULL, `createdAt` datetime NOT NULL, `updatedAt` datetime NOT NULL, PRIMARY KEY (`id`)) ENGINE=InnoDB AUTO INCREMENT=13 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

1 row in set (0.00 sec)

With Index EXPLAIN:

```
mysql> SHOW CREATE TABLE message;
+-----+
| Table | Create Table
+-----+
+-----+
| message | CREATE TABLE 'message' (
  'id' int NOT NULL AUTO INCREMENT,
  'senderId' int NOT NULL,
  'receiverId' int NOT NULL,
  'text' varchar(255) DEFAULT NULL,
  'createdAt' datetime NOT NULL,
  'updatedAt' datetime NOT NULL,
  PRIMARY KEY ('id'),
  KEY 'sender_receiver_id' (('senderId','receiverId','createdAt'))
) ENGINE=InnoDB AUTO INCREMENT=13 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci |
+-----+
1 row in set (0.00 sec)

mysql> EXPLAIN SELECT * FROM message WHERE (senderId = 1 AND receiverId = 2) OR (senderId = 2 AND receiverId = 1) ORDER BY createdAt ASC;
+-----+
| id | select_type | table | partitions | type | possible_keys | key | key_len | ref | rows | filtered | Extra
+-----+
| 1 | SIMPLE | message | NULL | range | sender_receiver_id | sender_receiver_id | 8 | NULL | 6 | 100.00 | Using index condition; Using filesort |
+-----+
1 row in set, 1 warning (0.00 sec)

mysql> EXPLAIN ANALYZE SELECT * FROM message WHERE (senderId = 1 AND receiverId = 2) OR (senderId = 2 AND receiverId = 1) ORDER BY createdAt ASC;
+-----+
| EXPLAIN
+-----+
+-----+
| -> Sort: message.createdAt (cost=3.21 rows=6) (actual time=0.135..0.137 rows=6 loops=1)
| -> Index range scan on message using sender_receiver_id over (senderId = 1 AND receiverId = 2) OR (senderId = 2 AND receiverId = 1), with index condition: (((message.receiverId = 2) and (message.senderId = 1)) or ((message.receiverId = 1) and (message.senderId = 2))) (cost=3.21 rows=6) (actual time=0.0518..0.113 rows=6 loops=1)
+-----+
1 row in set (0.00 sec)
```

Comparison Table:

	Without Index	Index
Type	ALL	range
Possible keys	NULL	sender_receiver_id
keys	NULL	sender_receiver_id
Key_len	NULL	8
rows	12	6
Filtered	8.3	100
Extra	Using WHERE, using filesort	Using index Condition, using filesort

Comparison Conclusion: With Index vs Without Index

Without Index:

- MySQL checked all 12 rows in the message table.
- type = ALL, which means a full table scan was performed.
- No index was used.
- It used Using where; Using filesort.
- This is slower when the table becomes large because MySQL has to check many rows.

With Composite Index (senderId, receiverId, createdAt):

- MySQL used the sender_receiver_id index.

- type changed from ALL to range.
- The estimated rows changed from 12 to 6.
- MySQL can directly find messages matching the senderId and receiverId conditions instead of checking the whole table.